

PRÁCTICA 2

Diseño de máquinas de estados



de

Raúl Escudero Jiménez

José Luis Pérez Martín

para la asignatura de

ELECTRÓNICA DIGITAL

Tutorizado por

Javier López García

1. DISEÑO DEL MÓDULO DE CONTROL	5
1.1. Diagrama de transición de estados	5
1.2. Tabla de transición de estados	6
1.2.1 Tabla de transición de estados: Versión ampliada:	7
1.3. Funciones Booleanas	8
1.4. Esquemático	9
My design	9
VCodAC.	10
Control	11
Control_combinacional	11
2. SIMULACIÓN DEL MÓDULO DE CONTROL	13
2.1. Vectores de Test	13
Test de apertura correcta y cierre con aborto	13
Test de apertura con aborto y cierre correcto	16
2.2. Cronogramas	17
2.3 Versión VHDL	18
CONCLUSIONES	21

1. DISEÑO DEL MÓDULO DE CONTROL

En este primer punto se explica el proceso de diseño del módulo exigido en esta práctica. Para ello se parte del diagrama de transición de estados, su visualización en una tabla para la posterior obtención de las funciones booleanas y diseño del circuito secuencial.

1.1. Diagrama de transición de estados

El sistema cuenta con un total de 5 estados posibles, cuya definición teórica, distinguiendo que hay dos usuarios posibles con secuencias distintas, es:

Estado A: Estado inicial del sistema, todavía no se pulsó ningún botón correcto. Corresponde a las variables de estado $EST(2:0) = 000$. La puerta está cerrada. Se llegará a "A" por tres medios distintos:

- Que se falle al pulsar el botón en cualquiera de los estados "A", "B", "C", es decir $R=1$.
- Inicio del sistema
- Completar la secuencia de cierre, es decir, acertar el botón una vez estás en el estado "E".

Estado B: Corresponde a las variables de estado $EST(2:0) = 001$. Se transiciona a este estado siempre que, una vez nos encontremos en el estado "A", se pulse el primer botón correcto en la secuencia de apertura. Es decir, se llega a B mediante el primer valor "1" de U.

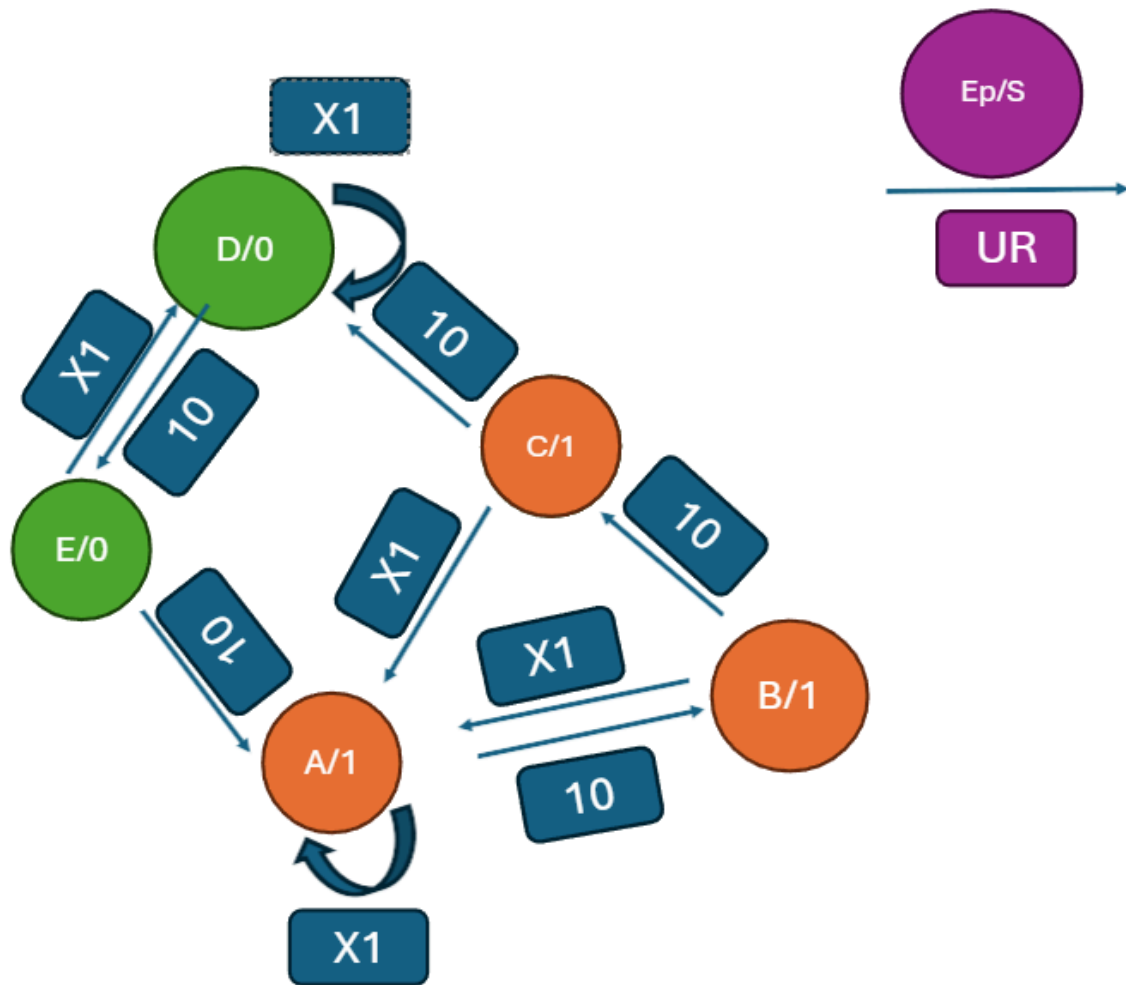
Estado C : Corresponde a las variables de estado $EST(2:0) = 011$. Se transiciona al estado C siempre que una vez estemos en el estado B se reciba el segundo valor "1" de U (siempre que no se haya fallado antes, en ese caso se regresaría de "B" a "A"). Se llegará a C siempre que se continúe con la secuencia de botones establecida.

Estado D: Corresponde a las variables de estado $EST(2:0) = 111$.

Este estado representa la primera etapa para la cual la puerta está abierta. Se puede acceder a D de dos maneras:

- Se completa la secuencia de apertura cuando estando en el estado "C" emitiendo el usuario un pulso de U con valor lógico "1", siendo este el tercero y último.
- Se falla en la secuencia de cierre, es decir, se obtiene $R=1$ en cualquiera de los estados "D", "E".

Estado E: Corresponde a las variables de estado $EST(2:0) = 110$. Se pulsó el primer botón correcto en la secuencia de cierre. Se llega a "E" únicamente cuando se recibe el primer valor "1" de U proveniente del estado "D".



1.2. Tabla de transición de estados

Tomaremos como la primera columna de la tabla el estado presente del sistema (Ep), siendo el resto de columnas la transición del sistema a los distintos estados posibles, mediante las distintas combinaciones de las variables U y R, es decir, las entradas del módulo de control. Por un lado, la señal U, toma origen en el módulo VCodAc, el cual, es un multiplexor que detecta cuando se pulsa el botón correcto en la secuencia de tanto apertura como cierre de la puerta, para luego ser introducida al módulo de control al detectar el bloque “p_fin”, un flanco de bajada del reloj.

Por otro lado la señal R es una puerta lógica OR entre “FC”, siendo “FC” el tiempo de operación que otorga el sistema para que el usuario introduzca la secuencia, y “F”, también proveniente del módulo “VCodAc” y que en su caso detecta cuando se ha errado en la pulsación del botón.

U detecta cuando se ha acertado el botón (U=1) y R cuando se ha agotado el tiempo o se ha fallado (R=1).

Ep	U=0 R=0	U=0 R=1	U=1 R=0	U=1 R=1
A	A	A	B	A
B	B	A	C	A
C	C	A	D	A
D	D	D	E	D
E	E	D	A	D

Podemos ver que la transición de los estados cuando UR= 01 y UR= 11 es la misma. Esto es debido a que en nuestro diseño tomamos en cuenta que la señal R tiene una mayor importancia que la señal U, llevando a que cualquier caso UR= X1 (donde X representa el valor “don't care”) sea tomado como fallo (R=1).

En otras palabras, se considera incorrecto activar el pulsador correcto (U=1) y otro erróneo (R=1) al mismo tiempo por motivos de seguridad.

Esto responde también a la pregunta planteada en la práctica acerca de la posibilidad de darle al botón correcto al mismo tiempo que el sistema completa el ciclo de conteo de “FC”. Si el tiempo prestado al usuario se agota y en ese exacto instante se pulsa el botón correcto, la secuencia se abortará.

1.2.1 Tabla de transición de estados: Versión ampliada:

Entradas: Las entradas del sistema son tanto U como R, definidas ya anteriormente.

Salidas: Tomamos como salida del sistema la variable S, la cual, controla el estado de la puerta en función de su valor (S=1 para puerta cerrada ,y, S=0 para puerta abierta).

Estado actual: Las variables q2,q1,q0 (p.ej: q2 q1 q0 = 000 => Estado A)

Estado presente: Las variables Q2,Q1,Q0, las cuales definen el progreso del sistema y proporcionan el valor “D” de los distintos 3 flip-flops a usar en la síntesis del sistema.

Al tener 5 estados posibles en el sistema, para cada combinación de UR tenemos 5 progresiones o transiciones del sistema distintas:

Caso 1: U=0; R=0:

U	R	q2	q1	q0	Q2	Q1	Q0	S
0	0	0	0	0	0	0	0	1
0	0	0	0	1	0	0	1	1
0	0	0	1	1	0	1	1	1
0	0	1	1	1	1	1	1	0
0	0	1	1	0	1	1	0	0

Caso 2: $U=X$; $R=1$;

U	R	q2	q1	q0	Q2	Q1	Q0	S
X	1	0	0	0	0	0	0	1
X	1	0	0	1	0	0	0	1
X	1	0	1	1	0	0	0	1
X	1	1	1	1	1	1	1	0
X	1	1	1	0	1	1	1	0

Caso 3: $U=1$; $R=0$;

U	R	q2	q1	q0	Q2	Q1	Q0	S
X	1	0	0	0	0	0	1	1
X	1	0	0	1	0	1	1	1
X	1	0	1	1	1	1	1	1
X	1	1	1	1	1	1	0	0
X	1	1	1	0	0	0	0	0

1.3. Funciones Booleanas

Los valores de las distintas D_i de los tres flip-flops son iguales a los valores de los Q_i en la tabla de transición de estados.

Obtenemos las funciones booleanas de los flip-flops mediante tablas de Karnaugh.

En la primera columna representaremos los valores de U, R, respectivamente, mientras que en la primera fila representaremos los valores de q_2, q_1, q_0 .

a) Determinación de D_0 :

D0	000	001	011	010	110	111	101	100
00		1	1		1			
01				1	1			
11				1	1			
10	1	1	1					

$$D_0 = R \cdot q_2 \cdot q_1 + U \cdot \text{NOT}(R) \cdot \text{NOT}(q_2) \cdot \text{NOT}(q_1) + \text{NOT}(R) \cdot q_0 \cdot \text{NOT}(q_2) + \text{NOT}(U) \cdot q_2 \cdot q_1 \cdot q_0$$

b) Determinación D1:

D1	000	001	011	010	110	111	101	100
00			1		1	1		
01					1	1		
11					1	1		
10		1	1			1		

$$D1 = U \cdot \text{NOT}(R) \cdot \text{NOT}(q2) \cdot q0 + \text{NOT}(R) \cdot q1 \cdot q0 + \text{NOT}(U) \cdot q2 \cdot q1 + R \cdot q2 \cdot q1$$

c) Determinación D2:

D0	000	001	011	010	110	111	101	100
00			1		1	1		
01					1	1		
11					1	1		
10			1			1		

$$D2 = \text{NOT}(R) \cdot U \cdot q1 \cdot q0 + \text{NOT}(U) \cdot q2 \cdot q1 + R \cdot q2 \cdot q1$$

Como la salida es “1” cuando la puerta está cerrada, tomará valor “1” siempre que se esté en cualquiera de los estados que asegura que la puerta no esté abierta:

$$S = \text{OR}(A+B+C) = \text{NOT}(D2) \cdot \text{NOT}(D1) \cdot \text{NOT}(D0) + \text{NOT}(D2) \cdot \text{NOT}(D1) \cdot D0 + \text{NOT}(D2) \cdot D1 \cdot D0$$

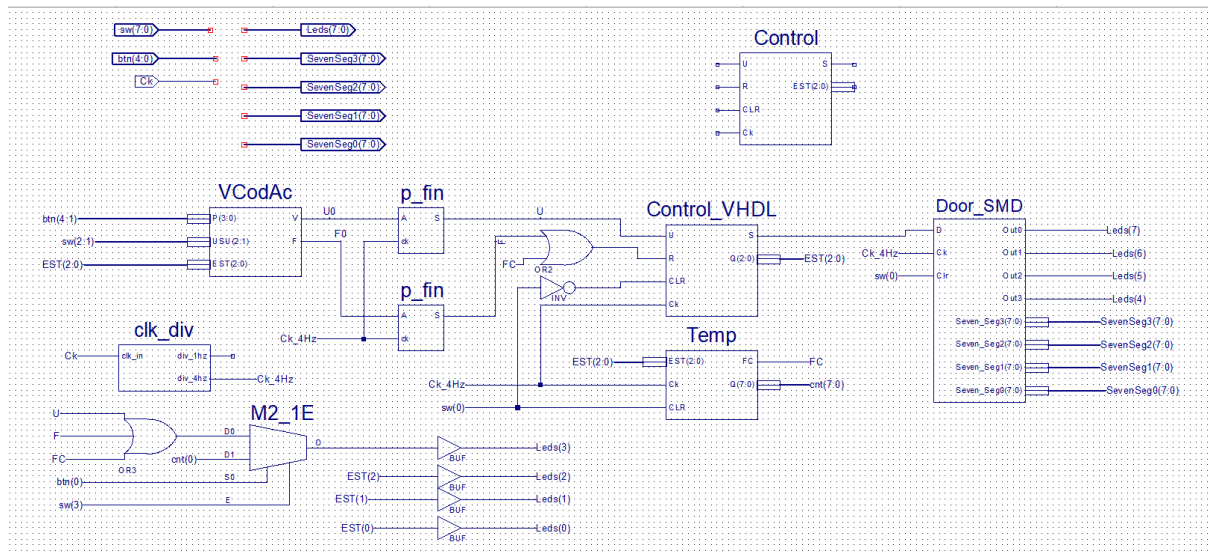
1.4. Esquemático

Esta sección presenta todos los módulos incluidos en la práctica

My design

Se incluye el módulo Control y su análogo en VHDL. Se tuvo que incluir un inversor en la entrada de CLEAR para facilitar el comportamiento del módulo control. No se incluyen Flips-Flops tipo D en las librerías de Xilinx IDE con entrada de Reset o Clear invertida, así que se ha utilizado uno ordinario.

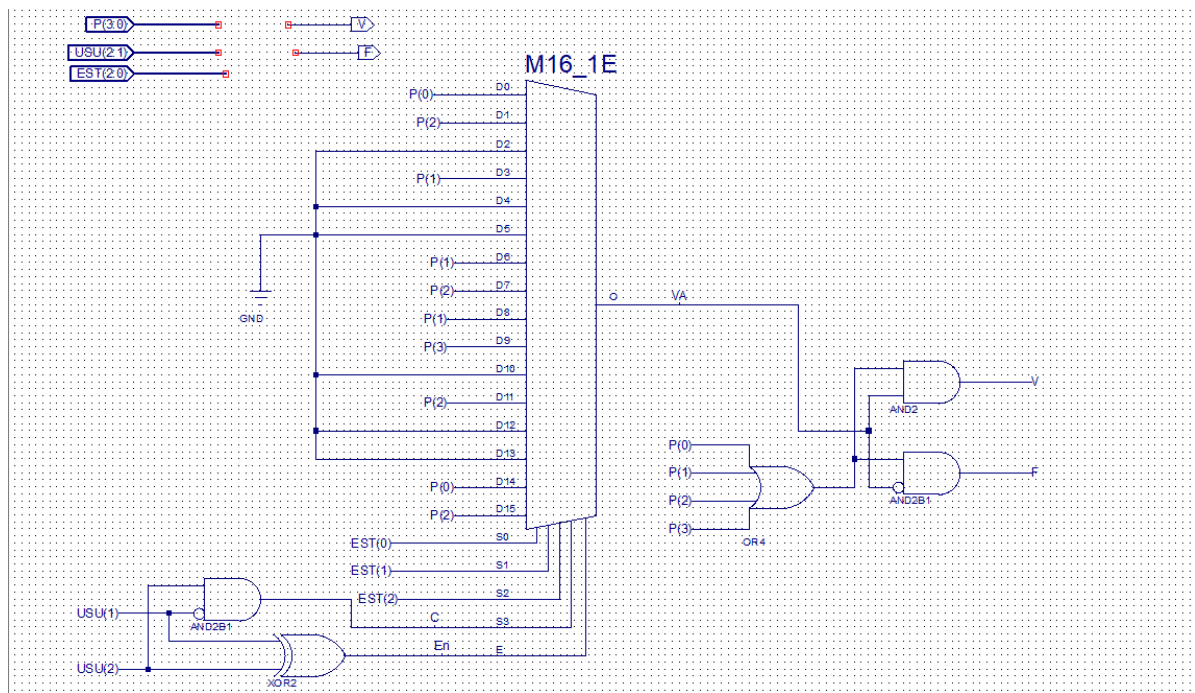
El sistema de seguridad de la puerta funciona con el switch (0) levantado (valor lógico 1), lo que corresponde a la entrada de CLEAR. Sin el inversor, los FFD no actualizarían el estado y permanecería en EST(2:0) = 000.



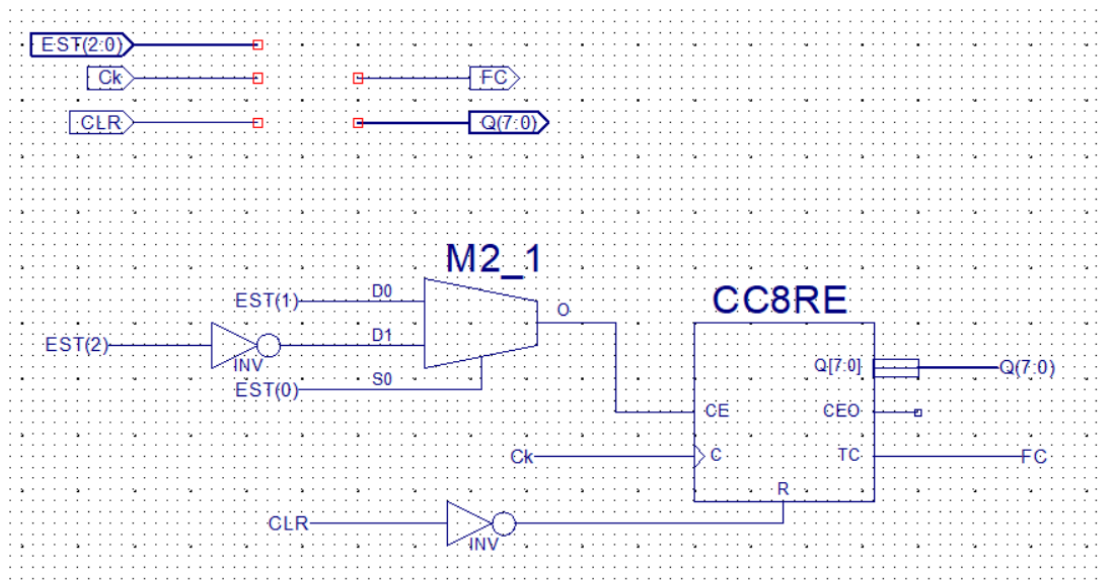
VCodAC.

En esta práctica, se propuso como tarea opcional para los grupos pares modificar la secuencia de cierre del usuario. Antes, se cerraba con la secuencia “btn(3)-btn(1)”; ahora con “btn(3)-btn(2)”. Para ello, se modificó la entrada D(6) de forma que entrase P(1) en lugar de P(0).

Se ha de aclarar que $\text{btn}(n) = P(n-1)$ —excluyendo a $n=0$ —, por lo que $\text{btn}(1) = P(0)$, $\text{btn}(2) = P(1)$, etc.

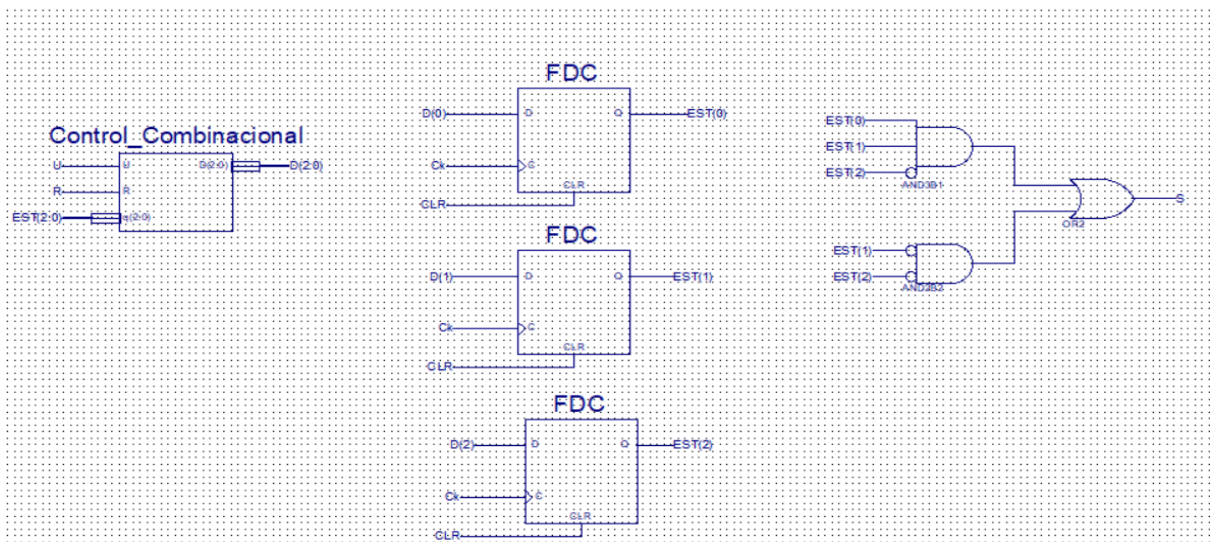


Temporizador



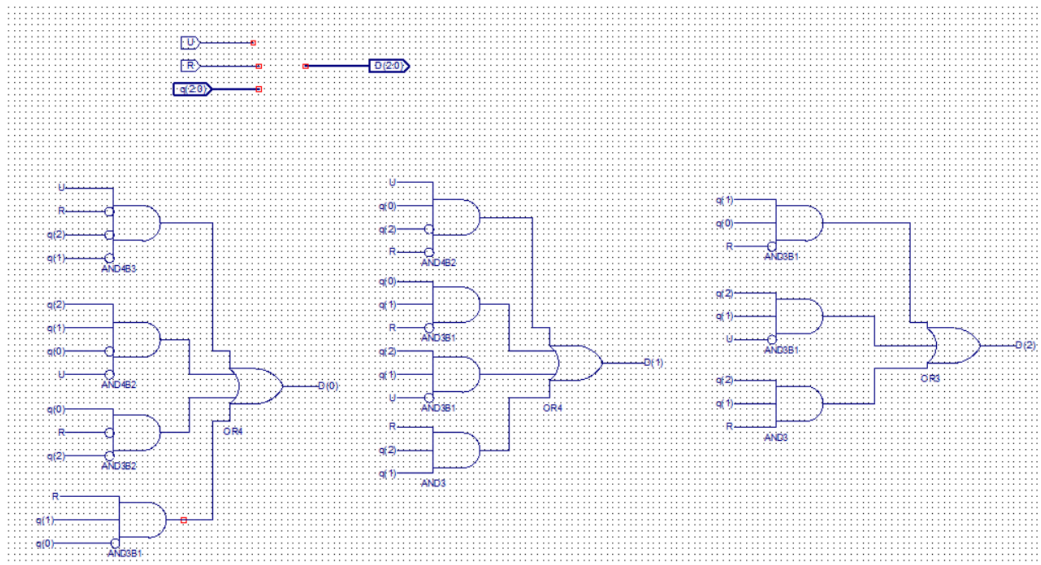
Control

El módulo Control es un sistema secuencial que incluye 3 Flip-Flops tipo D con entrada de Clear (necesarios para transitar entre los 5 estados definidos en nuestro sistema); un sistema combinacional que define el estado siguiente en combinación con el estado presente y las entradas; y otro sistema combinacional para la salida.



Control_combinacional

El circuito combinacional permite describir el estado siguiente (D(2:0)) con la combinación mediante puertas lógicas de las señales de entrada (U y R) y del estado presente (q(2:0)).



2. SIMULACIÓN DEL MÓDULO DE CONTROL

Primero se muestran los archivos de test y luego el resultado de simulación mediante cronogramas. Se decidió crear 2 cronogramas en lugar de dos para optimizar el trabajo de la práctica y apreciar mejor la evolución del sistema. Así, en cada test se incluye una secuencia de apertura sumada a una de cierre.

2.1. Vectores de Test

Para ambos vectores de test se ha seguido la misma metodología que en los cronogramas realizados en la práctica 1B. Para ello, se define una secuencia de señales con un “bucle for” y un “case”. Para cada iteración se sucede un ciclo de reloj.

Test de apertura correcta y cierre con aborto

En este test, se realiza la secuencia de apertura y cierre. La apertura se realiza sin errores y el cierre introduce un error en la secuencia, así que debe volver a introducirla desde el principio tras abortar la primera secuencia.

```
-- Vhdl test bench created from schematic
C:\Users\Usuario_UMA\Downloads\Practica2_JoseLuis_Raul_v2_24_05_26
\Control.sch - Mon May 25 13:35:52 2026
--
-- Notes:
-- 1) This testbench template has been automatically generated
using types
-- std_logic and std_logic_vector for the ports of the unit under
test.
-- Xilinx recommends that these types always be used for the
top-level
-- I/O of a design in order to guarantee that the testbench will
bind
-- correctly to the timing (post-route) simulation model.
-- 2) To use this template as your testbench, change the filename
to any
-- name of your choice with the extension .vhd, and use the
"Source->Add"
-- menu in Project Navigator to import the testbench. Then
-- edit the user defined section below, adding code to generate
the
-- stimulus for your design.
--
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;
LIBRARY UNISIM;
USE UNISIM.Vcomponents.ALL;
ENTITY Control_Control_sch_tb IS
END Control_Control_sch_tb;
```

ARCHITECTURE behavioral OF Control_Control_sch_tb IS
 constant CKsemiperiod : TIME := 20 ns;

```

COMPONENT Control
PORT(  Ck      :    IN      STD_LOGIC;
      CLR     :    IN      STD_LOGIC;
      R       :    IN      STD_LOGIC;
      U       :    IN      STD_LOGIC;
      S       :    OUT     STD_LOGIC;
      EST     :    OUT     STD_LOGIC_VECTOR (2 DOWNTO 0));
END COMPONENT;
```

```

SIGNAL Ck      :    STD_LOGIC;
SIGNAL CLR     :    STD_LOGIC;
SIGNAL R       :    STD_LOGIC;
SIGNAL U       :    STD_LOGIC;
SIGNAL S       :    STD_LOGIC;
SIGNAL EST     :    STD_LOGIC_VECTOR (2 DOWNTO 0);
```

BEGIN

```

UUT: Control PORT MAP(
  Ck => Ck,
  CLR => CLR,
  R => R,
  U => U,
  S => S,
  EST => EST
);
```

-- *** Test Bench - User Defined Section ***

tb : PROCESS

BEGIN

CLR <= '0';

R <= '0';

U <= '0';

for i in 0 to 16 loop

CK <= '1';

wait for CKsemiperiod;

case i is

--reset

when 1 =>

CLR <= '1';

when 2 =>

CLR <= '0';

```

-- apertura
    when 3 =>
        U <= '1';
    when 4 =>
        U <= '0';
    when 5 =>
        U <= '1';
    when 6 =>
        U <= '0';
    when 7 =>
        U <= '1';
    when 8 =>
        U <= '0';

    -- cierre
    when 9 =>
        U <= '1';
    when 10 =>
        U <= '0';
        when 11 =>
            R <= '1'; --Falla secuencia
        when 12 =>
            R <= '0';
    when 13 =>
        U <= '1';
        when 14 =>
            U <= '0';
    when 15 =>
        U <= '1';
        when 16 =>
            U <= '0';
    when others =>
        null;
end case;

    CK <= '0';
    wait for CKsemiperiod;
end loop;

    CK <= '1';
    wait for CKsemiperiod;
    WAIT; -- will wait forever
END PROCESS;
-- *** End Test Bench - User Defined Section ***
END;
```

Test de apertura con aborto y cierre correcto

La secuencia de apertura introduce un error (R=1), lo que reinicia la secuencia. Una vez introducida correctamente, se introduce la secuencia de cierre sin fallos y el sistema queda en el estado inicial de simulación (EST(2:0) = 000).

```
-- *** Test Bench - User Defined Section ***
tb : PROCESS
BEGIN
    CLR <= '0';
    R <= '0';
    U <= '0';

    for i in 0 to 18 loop
        CK <= '1';
        wait for CKsemiperiod;

        case i is
            --reset
                when 1 =>
                    CLR <= '1';
                when 2 =>
                    CLR <= '0';

            -- apertura con error
                when 3 =>
                    U <= '1';
                when 4 =>
                    U <= '0';
                when 5 =>
                    U <= '1';
                when 6 =>
                    U <= '0';
                when 7 =>
                    R <= '1'; -- error
                when 8 =>
                    R <= '0';

            -- apertura correcta
                when 9 =>
                    U <= '1';
                when 10 =>
                    U <= '0';
                when 11 =>
                    U <= '1';
```



```

        when 12 =>
            U <= '0';
        when 13 =>
            U <= '1';  -- error
        when 14 =>
            U <= '0';

            -- cierre
        when 15 =>
            U <= '1';
        when 16 =>
            U <= '0';
        when 17 =>
            U <= '1';
        when 18 =>
            U <= '0';
        when others =>
            null;
    end case;

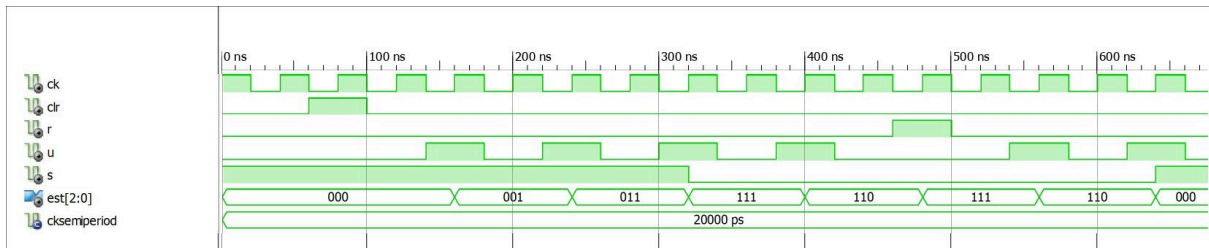
    CK <= '0';
    wait for CKsemiperiod;
end loop;

CK <= '1';
wait for CKsemiperiod;
WAIT; -- will wait forever
END PROCESS;
-- *** End Test Bench - User Defined Section ***
END;
```

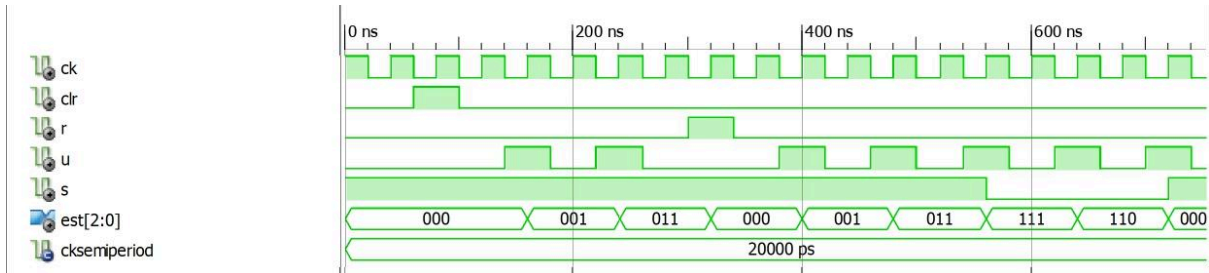
2.2. Cronogramas

Se presentan los cronogramas correspondientes a los dos vectores de test propuestos, cuyo funcionamiento es el previsto.

En el primer cronograma se comprueba que tres pulsos de “U” corresponden a una secuencia de apertura correcta, por lo que la puerta se abre. Por el contrario, en la secuencia de cierre se introduce un error (“R = 1”) y el sistema regresa al estado inicial de apertura (D, EST(2:0) = 111). Luego se introduce la secuencia correcta y se vuelve a cerrar.



En el segundo cronograma es en la secuencia de apertura donde se introduce un error y la operación se aborta; no obstante, la secuencia de cierre se lleva a cabo correctamente, sin errores.



2.3 Versión VHDL

En este último subapartado se presenta el módulo Control en lenguaje VHDL. Se realizó mediante lógica Case y exhibe el mismo comportamiento, tanto en la simulación en el laboratorio virtual como en la placa FPGA física.

```

-----
-----
-- Company:
-- Engineer:
--
-- Create Date:      16:46:47 05/24/2026
-- Design Name:
-- Module Name:      Control_VHDL - Behavioral
-- Project Name:
-- Target Devices:
-- Tool versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
-----
-----
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx primitives in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity Control_VHDL is
    Port ( U : in  STD_LOGIC;
          R : in  STD_LOGIC;
          CLR : in  STD_LOGIC;
          Ck : in  STD_LOGIC;
          S : out STD_LOGIC;
          Q : out STD_LOGIC_VECTOR (2 DOWNTO 0));
end Control_VHDL;

architecture Behavioral of Control_VHDL is
    signal EST_actual : STD_LOGIC_VECTOR (2 DOWNTO 0);
    signal EST_proximo : STD_LOGIC_VECTOR (2 DOWNTO 0);
begin

    Q <= EST_actual;
-- registro tipo D
FFD: process (CK, CLR) -- biestables tipo D
    begin
        if (CLR = '1') then
            EST_actual <= (others => '0');
        else
            if falling_edge(CK) then --Flanco bajada
                EST_actual <= EST_proximo;
            end if;
        end if;
    end process;

    process (U,R, EST_actual)
    begin
        --Apertura
        case EST_actual is
            when "000" => if (U = '0' and R= '0') then
EST_proximo <= "000"; end if;
                                                                    if (U = '1'
and R= '0') then EST_proximo <= "001"; end if;
                                                                    if (R='1')
then EST_proximo<= "000"; end if;

```

```

        when "001" => if (U = '0' and R= '0') then
EST_proximo <= "001"; end if;
                                                                    if (U = '1'
and R= '0') then EST_proximo <= "011"; end if;
                                                                    if (R='1')
then EST_proximo<= "000"; end if;

```

```

        when "011" => if (U = '0' and R= '0') then
EST_proximo <= "011"; end if;
                                                                    if (U = '1'
and R= '0') then EST_proximo <= "111"; end if;
                                                                    if (R='1')
then EST_proximo<= "000"; end if;

```

```

        --Cerrar
        when "111" => if (U = '0' and R= '0') then
EST_proximo <= "111"; end if;
                                                                    if (U = '1'
and R= '0') then EST_proximo <= "110"; end if;
                                                                    if (R='1')
then EST_proximo<= "111"; end if;

```

```

        when "110" => if (U = '0' and R= '0') then
EST_proximo <= "110"; end if;
                                                                    if (U = '1'
and R= '0') then EST_proximo <= "000"; end if;
                                                                    if (R='1')
then EST_proximo<= "111"; end if;

```

```

        when others => EST_proximo <= EST_actual;
        end case;
end process;

```

```

S <= (not EST_actual(2) and not EST_actual(1)) or (not
EST_actual(2) and EST_actual(1) and EST_actual(0));
end Behavioral;

```

CONCLUSIONES

Una vez concluida la explicación teórica de la práctica finalizamos el documento mediante un resumen de lo realizado. Durante la práctica, centrada exclusivamente en el módulo secuencial “Control”, conseguimos entender el funcionamiento de dicho módulo gracias a las herramientas dadas en las clases de teoría, permitiéndonos sacar el diagrama de transición de estados para luego poder obtener el resto de tablas de estado y funciones booleanas; igualmente, mediante el análisis del resto de módulos integrados en MyDesign.

Subsiguientemente, se desarrollaron los vectores test y se compararon los cronogramas para detectar posibles fallos en cualquier implementación no vistos con anterioridad. Esto nos permitió detectar la necesidad de incluir un inversor en la entrada de CLEAR del módulo control.

Tras verificar su correcto funcionamiento en simulación, se llevó a cabo la implementación del mismo módulo de control pero en lenguaje VHDL. Por último, se validó el comportamiento de ambas versiones en el laboratorio virtual del campus y en la placa FPGA del laboratorio físico.